

B.SC.CSIT 4th Semester
Operating System: CSC259

LAB 04

TITLE

FAMILIARIZATION WITH ROUND ROBIN PROCESS SCHEDULING ALGORITHM

OBJECTIVES

TO PERFORM ROUND ROBIN SCHEDULING ALGORITHM AND FINDING THE AVERAGE WAITING TIME AND TURNAROUND TIME

REQUIRED THEORY

Round Robin is a CPU scheduling algorithm where each process is assigned a fixed time slot in a cyclic way. It is simple, easy to implement, and starvation-free as all processes get fair share of CPU. One of the most commonly used technique in CPU scheduling as a core. It is preemptive as processes are assigned CPU only for a fixed slice of time at most. The disadvantage of it is more overhead of context switching.

SOURCE CODE

```
#include<stdio.h>

int main()
{
    // initialize the variable name
    int i, NOP, sum=0, count=0, y, quant, wt=0, tat=0, at[10], bt[10], temp[10];
    float avg_wt, avg_tat;

    printf(" Total number of process in the system: ");
    scanf("%d", &NOP);

    y = NOP; // Assign the number of process to variable y

    // Use for loop to enter the details of the process like Arrival time and the Burst Time
    for(i=0; i<NOP; i++) {
```

```

printf("\n Enter the Arrival and Burst time of the Process[%d]\n", i+1);

printf(" Arrival time is: \t"); // Accept arrival time

scanf("%d", &at[i]);

printf(" \nBurst time is: \t"); // Accept the Burst time

scanf("%d", &bt[i]);

temp[i] = bt[i]; // store the burst time in temp array

}

// Accept the Time qunat

printf("Enter the Time Quantum for the process: \t");

scanf("%d", &quant);

// Display the process No, burst time, Turn Around Time and the waiting time

printf("\n Process No \t\t Burst Time \t\t TAT \t\t Waiting Time ");

for(sum=0, i = 0; y!=0; ) {

if(temp[i] <= quant && temp[i] > 0) // define the conditions

{

    sum = sum + temp[i];

    temp[i] = 0;

    count=1;

}

else if(temp[i] > 0) {

    temp[i] = temp[i] - quant;

    sum = sum + quant;

}

if(temp[i]==0 && count==1) {

    y--; //decrement the process no.

}

printf("\nProcess No[%d] \t\t %d\t\t\t %d\t\t\t %d", i+1, bt[i], sum-at[i], sum-at[i]-bt[i]);

```

```

wt = wt+sum-at[i]-bt[i];

    tat = tat+sum-at[i];

    count =0;
}

if(i==NOP-1)  { i=0; }

else if(at[i+1]<=sum)

{ i++; }

else { i=0; }

}

// represents the average waiting time and Turn Around time

avg_wt = wt * 1.0/NOP;

avg_tat = tat * 1.0/NOP;

printf("\n Average Turn Around Time: \t%f", avg_wt);

printf("\n Average Waiting Time: \t%f", avg_tat);

return 0;

}

```

OUTPUT

```
ankit@ankit-virtual-machine:~$ ls
ankit  ankit.txt  Desktop  Documents  Downloads  lab02  Music  Pictures  Public  snap  Templates  Videos
ankit@ankit-virtual-machine:~$ mkdir lab04
ankit@ankit-virtual-machine:~$ cd lab04
ankit@ankit-virtual-machine:~/lab04$ touch process4.c
ankit@ankit-virtual-machine:~/lab04$ nano process4.c
ankit@ankit-virtual-machine:~/lab04$ ls
process4.c
ankit@ankit-virtual-machine:~/lab04$ gcc -o process process4.c
ankit@ankit-virtual-machine:~/lab04$ ls
process  process4.c
ankit@ankit-virtual-machine:~/lab04$ ./process
Total number of process in the system: 4

Enter the Arrival and Burst time of the Process[1]
Arrival time is:      9

Burst time is: 14

Enter the Arrival and Burst time of the Process[2]
Arrival time is:      0

Burst time is: 7

Enter the Arrival and Burst time of the Process[3]
Arrival time is:     14

Burst time is: 18

Enter the Arrival and Burst time of the Process[4]
Arrival time is:      3

Burst time is: 7
Enter the Time Quantum for the process:      3

  Process No      Burst Time      TAT      Waiting Time
Process No[2]      7            16            9
Process No[1]     14            24           10
Process No[4]      7            34           27
Process No[3]     18            32           14
Average Turn Around Time:      15.000000
Average Waiting Time:  26.500000ankit@ankit-virtual-machine:~/lab04$
```

CONCLUSION

HENCE WE STUDIED AND IMPLEMENTED ROUND ROBIN PROCESS SCHEDULING ALGORITHM IN LAB WHERE WE CALCULATED AVERAGE WAITING TIME AND AVERAGE TURN AROUND TIME OF PROCESSES IN UBUNTU OPERATING SYSTEM